

IMPORTANT: Taking the *Biz Directory* API and changing the database from businesses to another dataset BUT WITH NO OTHER CHANGES will pass, but no more than that. You need to show evidence of further development to score additional marks.

Finding a data set

The key to a good submission is to source a good data set. This can be for any application area, but you should be looking for data with at least 2 levels of information (for example businesses and reviews on businesses). The volume of data is not so important, but you may want to demonstrate pagination, searching and sorting – so a dataset that supports these would be best.

Although you will need the data to be in JSON format, you may find data in another format such as CSV or XML. There are good online tools that will convert between data types, such as that seen at <http://www.convertcsv.com/csv-to-json.htm>.

There are lots of good online sources for useful datasets. You will easily find these by searching, but some starting points are presented in the list provided in the Dataset Preparation document.

If you find a good dataset that does not contain a 2nd level of data (e.g. “reviews” in the “business” dataset, it is easy to add one either by writing a short piece of Python code (as shown in Practical BE07), or by using the *Find* and *Replace* functions in a text editor. For example, if your dataset was a list of holiday destinations such as

```
[
  {
    "location": "Paris",
    "country" : "France",
    "language" : "French"
  },
  {
    "location": "Los Angeles",
    "country" : "USA",
    "language" : "English"
  },
  ...
]
```

you could add a 2nd level “comments” field by doing a *find* operation for `}` (highlighted above) and a *replace* with `, "comments" : [ ] },`. This would result in the dataset being amended as follows.

```
[
  {
    "location": "Paris",
    "country" : "France",
    "language" : "French",
    "comments": []
  },
  {
    "location": "Los Angeles",
    "country" : "USA",
    "language" : "English",
    "comments": []
  },
  ...
]
```

Note that this will not change the last element in the dataset as there is no comma after the closing }, but this one can be edited by hand.

Another potential issue with some datasets is that they aren't structured as a JSON Array but include the array as one of their fields. Let's say our holiday dataset from the previous example looked like this

```
{
  "owner" : "Adrian",
  "season": "2025",
  "destinations" : [
    {
      "location": "Paris",
      "country" : "France",
      "language" : "French"
    },
    {
      "location": "Los Angeles",
      "country" : "USA",
      "language" : "English"
    },
    ...
  ]
}
```

Here, the dataset consists of a single object, but the "destinations" element is the JSON array that we need. Again, the easiest solution is a text editor, where we would simply delete all of the text highlighted in green and save what remains.

If you have any additional dataset formatting queries that aren't covered by these, please get in touch.

API Functionality Tips

Note: It is absolutely not necessary to cover all of these – it is just a range of suggestions that might be useful if you are struggling to find ways to improve on my basic implementation.

The best submission will be an application written from scratch. Obviously, you can use lessons learned from the *Biz Directory* application, but the selection of API endpoints provided, and the front-end interface should be one tailored for your own data set.

Alternatively, you can take the *Biz Directory* application as a starting point, change the database and enhance the functionality. There are many things that are deliberately omitted from the sample application, in order to leave space for you to work in. These include...

- Database queries that are specific to your dataset. The standard CRUD (Create, Retrieve, Update, Delete) operations are important, but dataset-specific queries will greatly improve the usability of your API. For example, if your dataset is a list of Olympic medalists through history, useful queries might be to obtain the most decorated athletes of all time, athletes that have won gold medals in successive games, athletes who have won medals in different events, and so on. Think about your dataset and what the most information that can be extracted from it.
- More complex database queries. We have demonstrated fetching a business by `_id`, but what about by `city` (or equivalent in your dataset)? That would return a collection of results that you would need to iterate across and display. You could also show multi-criteria queries, such as businesses in Belfast with category “Restaurant”. We have also studied the Aggregation Pipeline that enables complex queries, including those based on geographic location.
- A “search” endpoint would be very useful in some applications. For example, in the *Biz Directory* application, you could search for a business by name, location, keyword or rating – or some combination of these.
- Some of the “Try it now!” exercises have had their solution withheld so that you can gain credit for it in your applications. For example, when editing an object, the current version requires that all fields are provided. A more flexible solution would be to allow only a subset of fields to be provided – those fields for which new values are not provided should remain with their current values.
- Currently, reviews for a business are returned in a single list – no matter how many of them are stored in the database. It would be better to provide pagination for reviews – as well as the ability to sort reviews by date or rating (either ascending or descending)

Please note that this is NOT a checklist of features that you MUST include – but ideas on how you can raise the level of your own application. There will be many other opportunities for enhanced functionality depending on the nature of the dataset that you choose to work with.

Development Approach

The best approach is to tackle the API as follows:

i) **Plan**

Find a dataset and think carefully about the operations you want to provide in it. Is it sensible to implement add, edit and delete operations on the core data elements? For example, if your dataset is details of “Game Of Thrones” episodes, then add/edit/delete probably aren’t appropriate – in this case, you would demonstrate that functionality on comments or reviews left by users of your application, but you should include all CRUD operations somewhere within your functionality.

A good approach is to create a list of the database operations required with their HTTP verb. For example...

GET /episodes	retrieve list of episodes
GET /episodes/<id>	retrieve a specific episode
GET /episodes/<id>/reviews	retrieve the reviews for a specific episode
POST /episodes/<id>/reviews	add a review for an episode

etc...

Make sure that your API follows the design principles – use nouns in the URL rather than verbs, and be consistent in the URL patterns implemented.

ii) **Back-end**

Build the back-end API to support the operations identified in Stage 1. Think carefully about the format of the data returned from the API. For example, in the “Game of Thrones” route **GET /episodes** you might return additional data that would be useful to the front end, such as the structure below which returns the page being returned and the total number of pages as well as the data for the current page.

```
{
  "this_page" : 1,
  "number_of_pages" : 10
  "data" : [
    {
      "episode_number" : 1,
      "air_date" : "2019-10-01",
      "title" : "Episode Title",
      "synopsis" : "A description...",
      "comments" []
    },
    {
      ...
    }
  ]
}
```